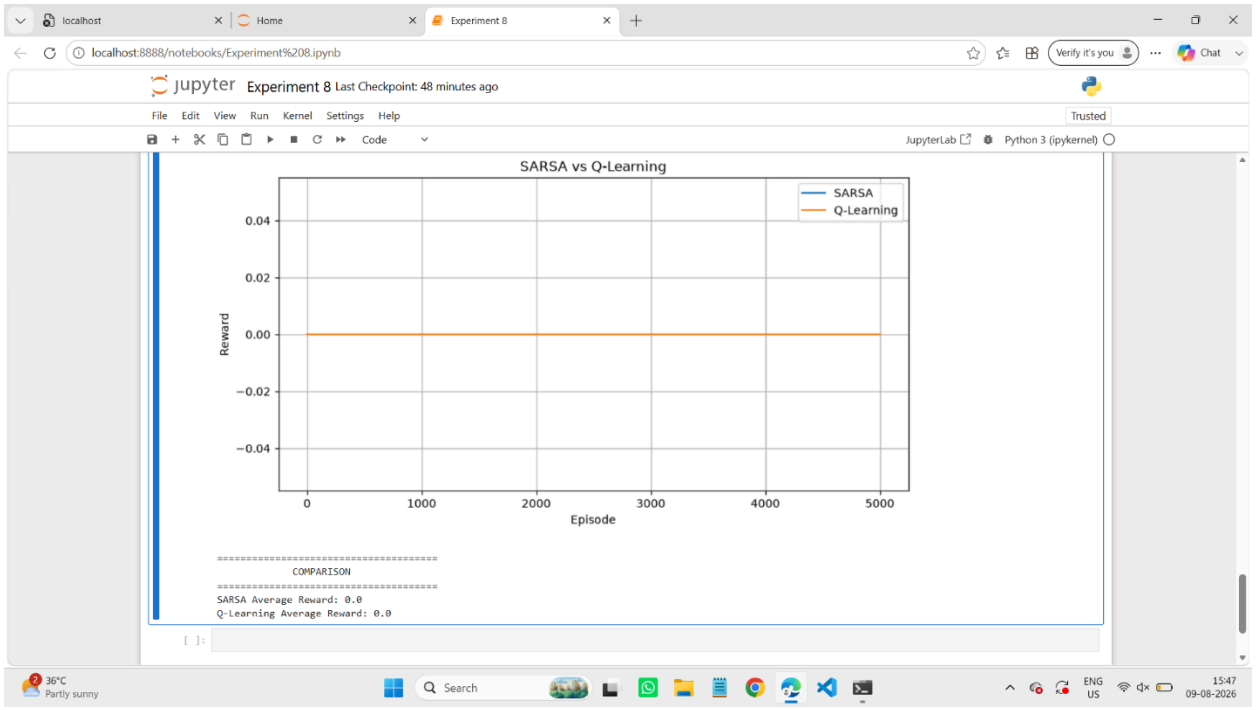
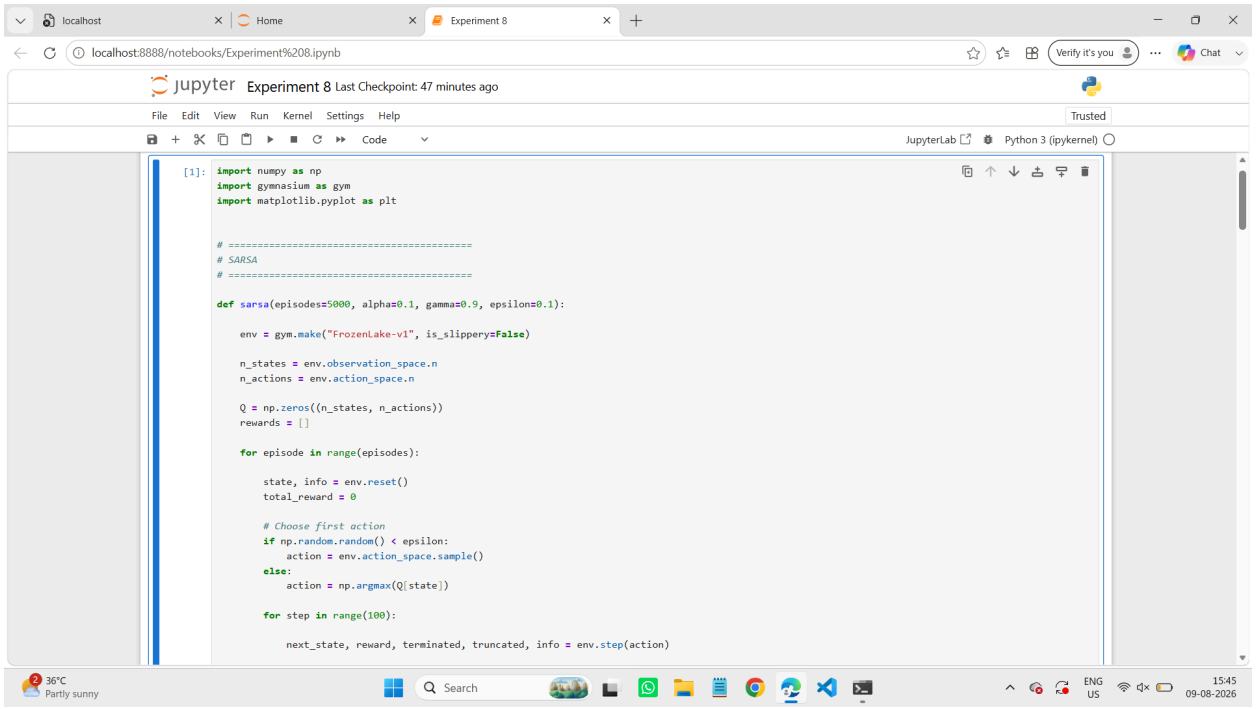
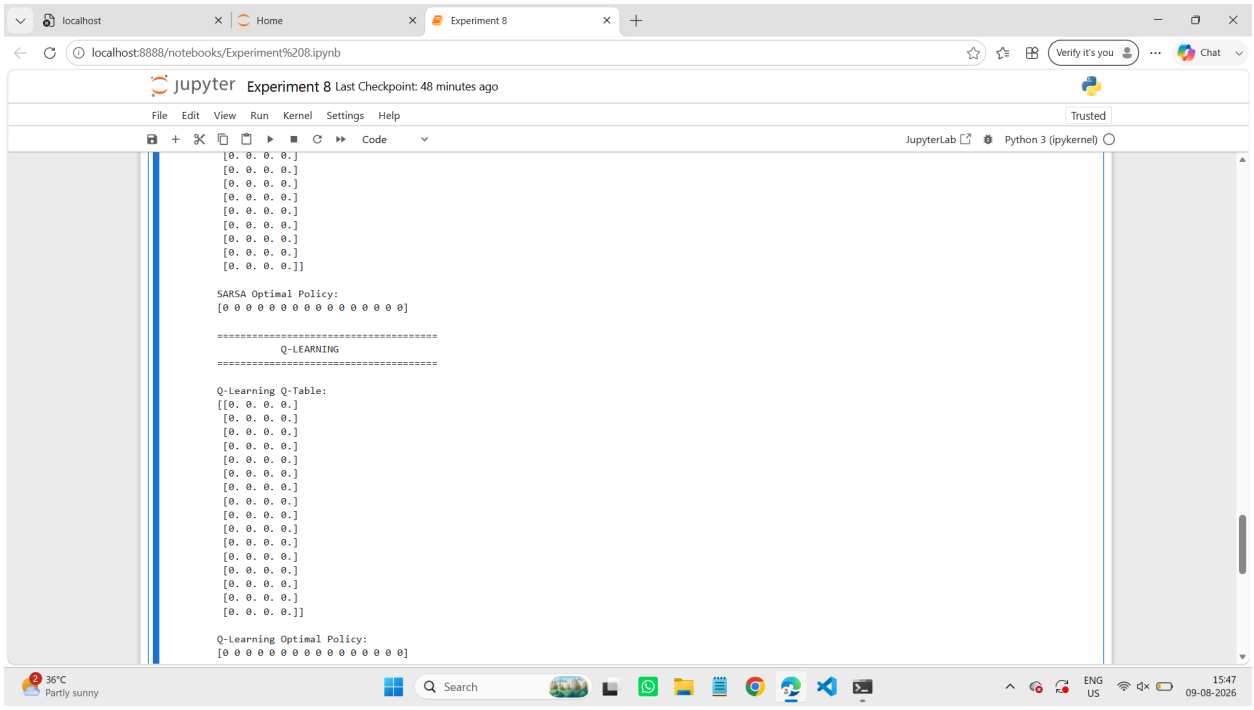
Experiment 8 RL

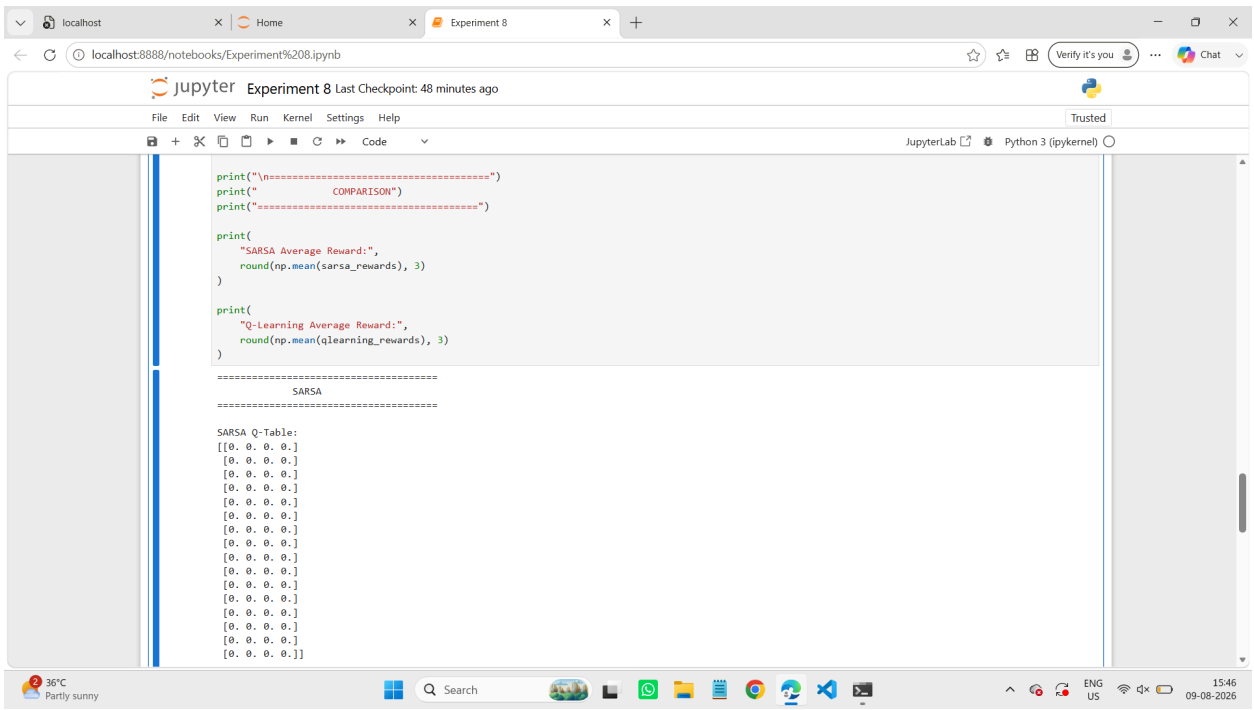
Name: G.Vinay Kumar Reddy **Reg No:** 192472093

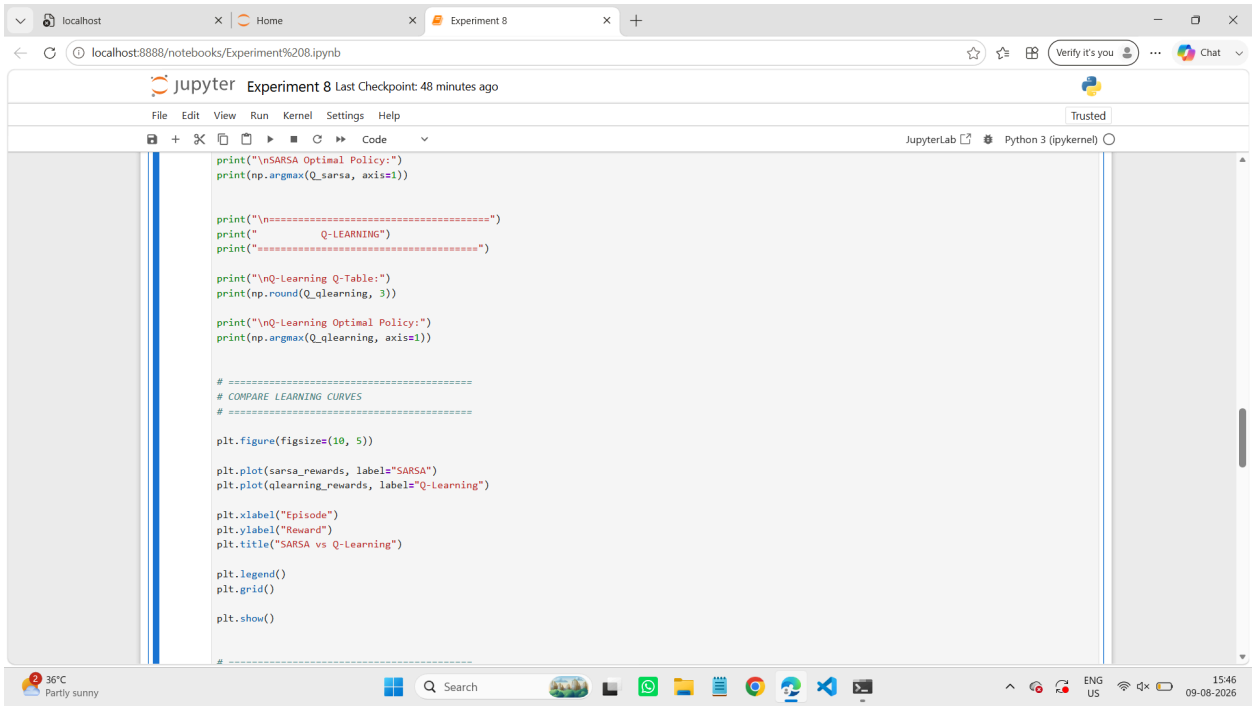
Course Code: MLA0305 **Slot:** B



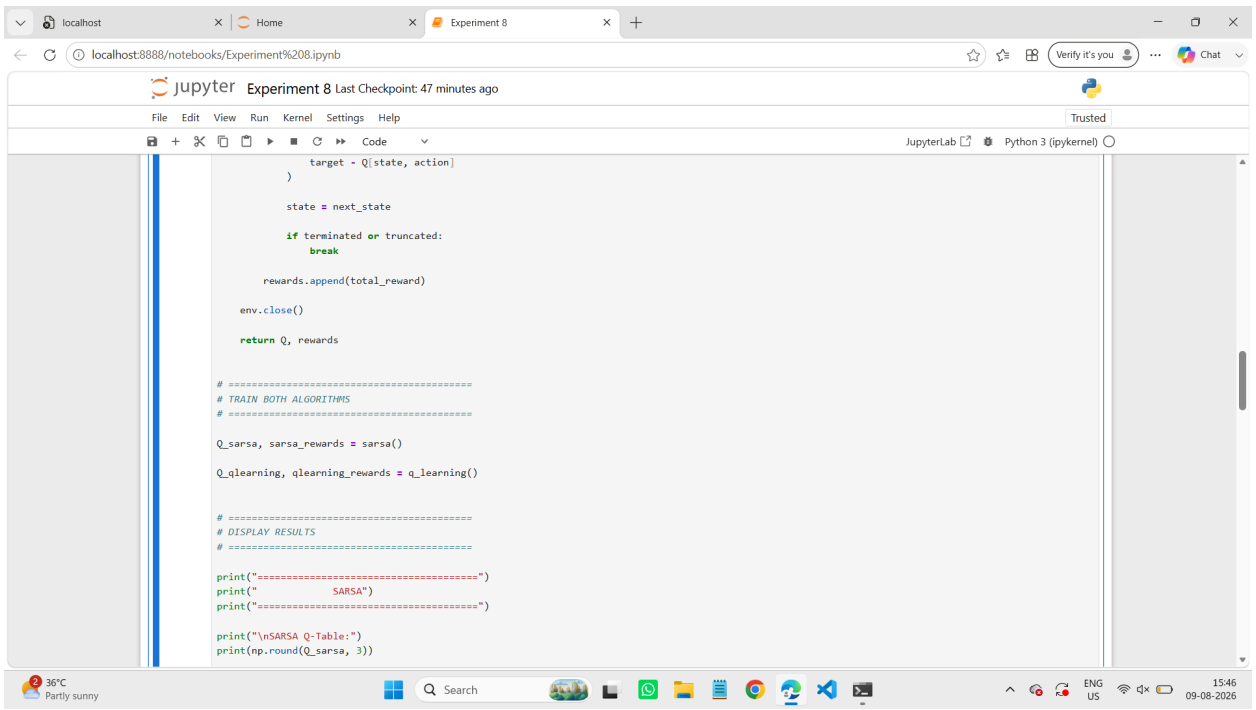



localhost
Home
Experiment 8
localhost:8888/notebooks/Experiment%208.ipynb
jupyter Experiment 8 Last Checkpoint: 48 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]]
SARSA Optimal Policy:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
==========
Q-LEARNING
==========
Q-Learning Q-Table:
[[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]]
Q-Learning Optimal Policy:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
36°C
Partly sunny
Search
ENG
US
15:47
09-08-2026

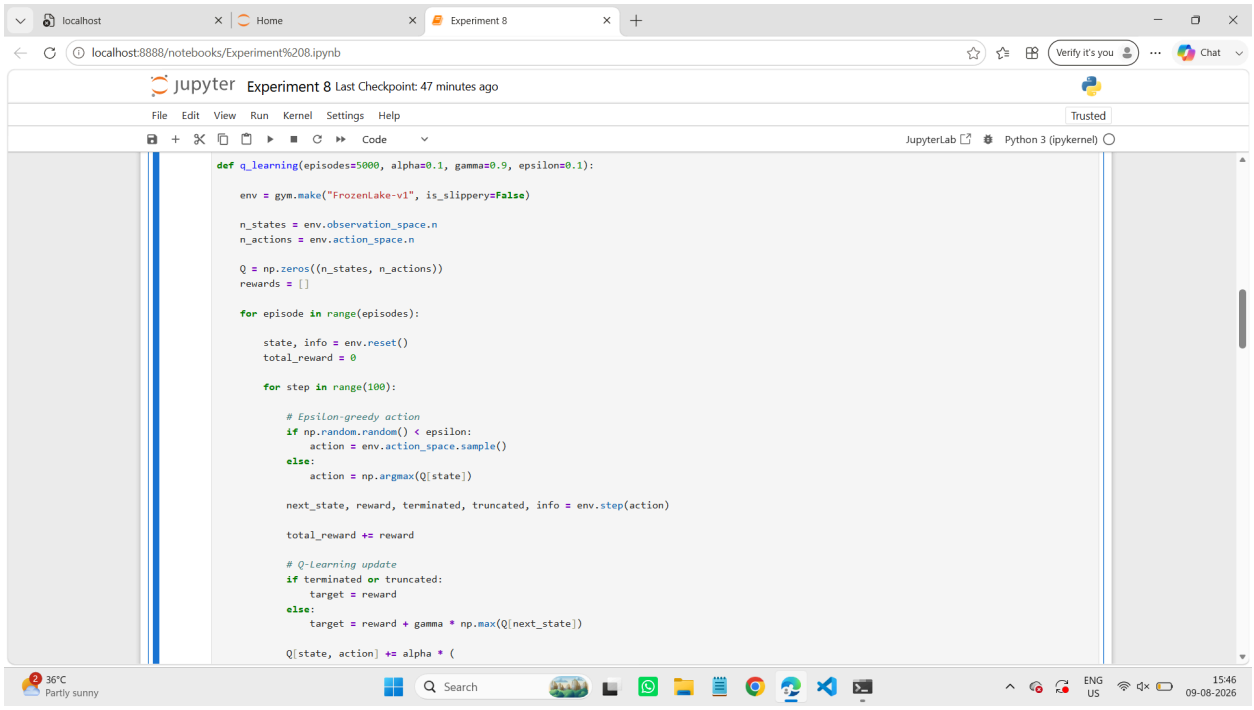

localhost
Home
Experiment 8
localhost:8888/notebooks/Experiment%208.ipynb
jupyter Experiment 8 Last Checkpoint: 48 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
print("\n==========")
print(" COMPARISON")
print("==========")
print(
"SARSA Average Reward:",
round(np.mean(sarsa_rewards), 3)
)
print(
"Q-Learning Average Reward:",
round(np.mean(qlearning_rewards), 3)
)
==========
SARSA
==========
SARSA Q-Table:
[[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]]
36°C
Partly sunny
Search
ENG
US
15:46
09-08-2026



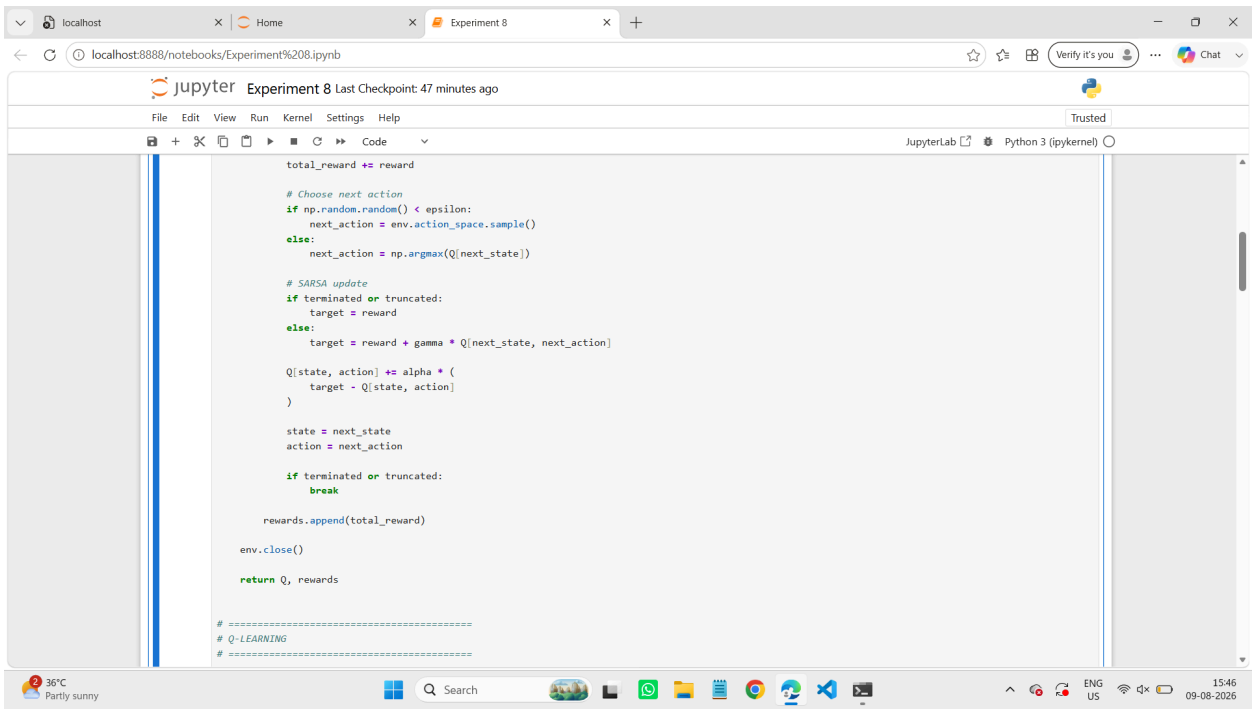
```
print("\nSARSA Optimal Policy:")
print(np.argmax(Q_sarsa, axis=1))


print("\n========================================")
print("           Q-LEARNING")
print("========================================")

print("\nQ-Learning Q-Table:")
print(np.round(Q_qlearning, 3))

print("\nQ-Learning Optimal Policy:")
print(np.argmax(Q_qlearning, axis=1))


# ============================================
# COMPARE LEARNING CURVES
# ============================================

plt.figure(figsize=(10, 5))

plt.plot(sarsa_rewards, label="SARSA")
plt.plot(qlearning_rewards, label="Q-Learning")

plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("SARSA vs Q-Learning")

plt.legend()
plt.grid()

plt.show()


# ============================================
```



```
                target - Q[state, action]
            )

            state = next_state

            if terminated or truncated:
                break

        rewards.append(total_reward)

    env.close()

    return Q, rewards


# ============================================
# TRAIN BOTH ALGORITHMS
# ============================================

Q_sarsa, sarsa_rewards = sarsa()

Q_qlearning, qlearning_rewards = q_learning()


# ============================================
# DISPLAY RESULTS
# ============================================

print("========================================")
print("              SARSA")
print("========================================")

print("\nSARSA Q-Table:")
print(np.round(Q_sarsa, 3))
```



```
def q_learning(episodes=5000, alpha=0.1, gamma=0.9, epsilon=0.1):

    env = gym.make("FrozenLake-v1", is_slippery=False)

    n_states = env.observation_space.n
    n_actions = env.action_space.n

    Q = np.zeros((n_states, n_actions))
    rewards = []

    for episode in range(episodes):

        state, info = env.reset()
        total_reward = 0

        for step in range(100):

            # Epsilon-greedy action
            if np.random.random() < epsilon:
                action = env.action_space.sample()
            else:
                action = np.argmax(Q[state])

            next_state, reward, terminated, truncated, info = env.step(action)

            total_reward += reward

            # Q-Learning update
            if terminated or truncated:
                target = reward
            else:
                target = reward + gamma * np.max(Q[next_state])

            Q[state, action] += alpha * (
```



```
                total_reward += reward

                # Choose next action
                if np.random.random() < epsilon:
                    next_action = env.action_space.sample()
                else:
                    next_action = np.argmax(Q[next_state])

                # SARSA update
                if terminated or truncated:
                    target = reward
                else:
                    target = reward + gamma * Q[next_state, next_action]

                Q[state, action] += alpha * (
                    target - Q[state, action]
                )

                state = next_state
                action = next_action

                if terminated or truncated:
                    break

            rewards.append(total_reward)

        env.close()

        return Q, rewards


# ========================================
# Q-LEARNING
# ========================================
```